

VERISPRINT

The Proof-of-Work Platform for Software Teams

Full platform specification — production release

Master Specification — v3 (Production)

Competitor Feature Parity · Authentication & Roles · Super-Admin Dashboard

Public Marketing Site · Full PRD · Architecture · Data Model · Build Brief

Prepared: August 2026 — Revision 3

Supersedes Master Spec v2. Adds full competitive feature parity with Jellyfish, LinearB, GitClear, Swarmia, and Allstacks, plus authentication, super-admin operations, and public site scope.

Confidential — internal planning use.

Table of Contents

- 1. Executive Summary (PRP)
- 2. Differentiation Strategy
- 3. Competitor Feature Parity Matrix (Jellyfish, LinearB, GitClear, Swarmia, Allstacks)
- 4. Personas & Core Use Cases
- 5. Full Product Requirements (PRD) — Complete Feature Set by Phase
- 6. Authentication & Access Control
- 7. Super-Admin Dashboard (Platform Operator View)
- 8. Public Pages & Marketing Site
- 9. Workflows & System Flows
- 10. Production Architecture
- 11. Core Data Model
- 12. Pricing & Packaging
- 13. Build Brief for Engineering
- 14. Meeting Talking Points

1. Executive Summary (PRP)

VeriSprint is a GitHub-connected platform that reads every commit and pull request in depth and turns raw code activity into a verified, plain-language account of what was actually built — reconciled against what was claimed in standups, tickets, and status updates. This revision brings VeriSprint to full production scope: complete feature parity with the established engineering-intelligence category (Jellyfish, LinearB, GitClear, Swarmia, Allstacks), plus the unique verification layer that none of them offer, plus the operational infrastructure — authentication, a super-admin control plane, and a public site — needed to run this as a real product rather than a demo.

One-line pitch

"VeriSprint tells you, in plain English, what your engineering team actually shipped — and proves it."

What's new in this revision

- A full feature parity matrix against Jellyfish, LinearB, GitClear, Swarmia, and Allstacks, so nothing from the established category is missing.
- Authentication and role-based access control across five roles, including a client-facing portal role.
- A Super-Admin Dashboard — the founder/operator's own control plane for usage, errors, billing, and system health across every customer workspace.
- A public marketing site scope, with a direct answer to whether it's still needed now that this is production (short answer: yes — see Section 8).

2. Differentiation Strategy

Matching competitor features closes gaps; it does not by itself create a reason to switch. The four pillars below are what still make this genuinely VeriSprint's own product, not a smaller bundle of the category:

- **Evidence-first judgment, not vibes-based scoring.** Every score is traceable to cited evidence (tests, call graph, TODOs, acceptance criteria match) — inspectable, not a black box.
- **Developer-first framing, not manager-first.** The tool saves developers time first (auto-drafted standups, onboarding docs); verification for PMs is a byproduct of a tool developers actually like.
- **Outward-facing, not just internal.** The same engine faces clients (agency proof-of-work portal), investors (update generator), and new hires (onboarding docs) — audiences the competitor category never touches.
- **Narrow, fast, self-serve.** No sales call, no multi-week rollout — the buyer priced out of Jellyfish's enterprise contracts gets value the same day.

3. Competitor Feature Parity Matrix

Every signature capability across the five named platforms, and where it now lives in VeriSprint. This is the checklist that proves nothing from the category was left out.

Jellyfish — Engineering ↔ Business Alignment

Jellyfish capability	VeriSprint equivalent	Phase
Investment Allocation (work mapped to strategic initiatives vs BAU)	Initiative Allocation View — commits/tickets tagged and rolled up by business initiative	V2
Resource allocation across people/teams/priorities	Team Allocation Dashboard	V2
Executive dashboards with drilldown to team/repo/initiative	Executive Rollup — PM Dashboard drilldown to person/ticket/repo	MVP/V2
R&D cost reporting for financial/audit purposes	Cost Capitalization Report (audit-ready R&D spend by initiative)	V3
SSO (Okta) and enterprise alerting integrations	SSO/SAML, see Section 6	V3

LinearB — Workflow Automation & DORA

LinearB capability	VeriSprint equivalent	Phase
DORA metrics (deployment frequency, lead time, CFR, MTTR)	DORA Metrics Panel, benchmarked against team history	V2
WorkerB bot — Slack/Teams nudges when PRs stall	Blocker Nudge Bot — Slack alerts for stalled PRs/reviews	V2
gitStream — auto-assign reviewers, auto-label PRs, auto-merge low-risk changes	PR AutoRoute — policy-based reviewer assignment and labeling	V3
Team Goals tied to DORA/performance signals	Team Goals & Targets module	V2
AI tool usage tracking (Copilot, Cursor, Claude Code)	AI Contribution Tracker — see Section 5	V2

GitClear — Code Quality & Diff Analysis

GitClear capability	VeriSprint equivalent	Phase
Diff Delta — durable change vs churn/rework, filters out noise	Durable Change Score — extends the Evidence Ledger with a churn-vs-real-progress metric	V2
Code quality stats (churn, duplicate blocks, rework rate)	Code Health Signals, feeding the Confidence Score	V2

GitClear capability	VeriSprint equivalent	Phase
Snap Changelogs — AI-generated visual change snapshots	Visual Changelog — auto-generated per sprint/PR, stakeholder-facing	V2
AI-generated vs human-authored code split	AI Contribution Tracker (per-commit AI/human attribution)	V2
On-prem edition for security-sensitive customers	Private/On-Prem LLM Option, see Phase 3 (V3)	V3

Swarmia — Developer Experience & Working Agreements

Swarmia capability	VeriSprint equivalent	Phase
Developer experience (DX) surveys correlated with delivery metrics	Pulse Surveys — short DX check-ins linked to Confidence/Evidence data	V3
Working agreements (team-defined norms, tracked automatically)	Working Agreements module — e.g. PR size limits, review SLAs, tracked and flagged	V3
AI-assisted PR detection across Copilot/Cursor/Claude Code, incl. cloud agents	AI Contribution Tracker, extended to cloud-agent PR detection	V3
Slack feedback loops	Slack digests + Blocker Nudge Bot (shared infrastructure)	MVP/V2
Self-serve, transparent per-seat pricing	Adopted directly — see Section 12	MVP

Allstacks — Forecasting & Risk

Allstacks capability	VeriSprint equivalent	Phase
ML-based delivery forecasting (predicted completion dates)	Delivery Forecast — ML model trained on the workspace's own Evidence Ledger history	V3
Automated risk alerts before deadlines slip	Risk Radar — extends Ticket Drift & Orphan Commit detectors into a proactive alert feed	V2/V3
Value stream mapping (idea to production)	Value Stream View	V3
Automated software cost capitalization, audit-ready	Cost Capitalization Report (shared with Jellyfish parity item)	V3
200+ data source connectors	Open Integration Framework — GitHub/GitLab/Bitbucket, Jira/Linear, Slack at launch, connector SDK for more	V2/V3

Net result

Every headline capability across all five platforms now has a home in the VeriSprint roadmap, phased so the verification-specific features (Evidence Ledger, Claimed vs Shipped, Repo Chat, Client Portal) still ship first and remain the product's identity — the competitor-parity features extend the platform outward from that core rather than replacing it.

4. Personas & Core Use Cases

Persona	Core need	Primary feature
Non-technical founder	Know if progress is real before the next investor update	Investor Update Generator, Delivery Forecast
Product manager	Verify standup claims without reading code	Claimed vs Shipped, Risk Radar, Repo Chat
Agency owner	Prove billed hours match delivered work	Client Proof-of-Work Portal
Engineering manager	Spot stuck work, manage DORA/team health	Blocker Nudge Bot, DORA Panel, Working Agreements
CTO / VP Engineering	Report investment allocation and cost to the board	Investment Allocation, Cost Capitalization
Developer	Stop re-explaining work; get credit for real progress	Auto-Drafted Standup, Personal Evidence Ledger
New hire	Understand an unfamiliar codebase fast	Onboarding Doc Generator
Platform operator (you)	Know the system is healthy and customers are succeeding	Super-Admin Dashboard

5. Full Product Requirements (PRD) — Complete Feature Set by Phase

Consolidated: original VeriSprint-unique features, the advanced feature set, and the newly mapped competitor-parity features.

Phase 1 — MVP (already built / demoed)

- GitHub connection (OAuth + App install, read-only scopes)
- Deep commit/PR analysis with plain-English summaries
- Evidence Ledger (cited evidence behind every judgment)
- Completion-confidence score per ticket
- Auto-drafted standup update
- Claimed vs Shipped view
- Mismatch alerts (framed as questions)
- PM dashboard and Developer view

Phase 2 — Early paid tier

- AI Repo Chat ("did we ship X this week?")
- Sprint rollup report
- Slack/email digests
- Historical accuracy score
- Ticket Drift Detector & Orphan Commit Detector
- Cross-file impact map
- DORA Metrics Panel (*LinearB parity*)
- Blocker Nudge Bot (*LinearB parity*)
- Team Goals & Targets (*LinearB parity*)
- Durable Change Score & Code Health Signals (*GitClear parity*)
- Visual Changelog (*GitClear parity*)
- AI Contribution Tracker — AI vs human authored code (*GitClear/LinearB/Swarmia parity*)
- Investment Allocation & Team Allocation Dashboard (*Jellyfish parity*)
- Risk Radar (proactive delivery-risk alert feed) (*Allstacks parity*)
- Open Integration Framework (GitHub/GitLab/Bitbucket, Jira/Linear, Slack)

Phase 3 — Growth / enterprise expansion

- Client Proof-of-Work Portal (white-labeled)
- Investor Update Generator
- Confidence-Weighted Burndown

- Onboarding Doc Generator
- Async Standup Replacement + ROI/Time-Saved Calculator
- Anomaly check-in nudges
- Multi-repo / monorepo intelligence
- Private / On-Prem LLM option (*GitClear parity*)
- Compliance & Audit Trail mode
- PR AutoRoute — policy-based reviewer assignment/auto-merge (*LinearB gitStream parity*)
- Pulse Surveys & Working Agreements (*Swarmia parity*)
- Delivery Forecast (ML-based completion prediction) (*Allstacks parity*)
- Value Stream View (*Allstacks parity*)
- Cost Capitalization Report, audit-ready (*Jellyfish/Allstacks parity*)
- SSO/SAML, enterprise alerting integrations (*Jellyfish parity*)

6. Authentication & Access Control

Production requires real auth, not a shared login. Five roles, least-privilege by default:

Role	Access	Notes
Super Admin (you)	Full platform control across every workspace	Internal only. Powers the Super-Admin Dashboard (Section 7). Never assigned to a customer.
Workspace Owner/Admin	Full control within their own workspace: billing, integrations, user management	Typically the founder/PM who signed up.
Manager/PM	Full read access to dashboards, alerts, reports within their workspace; cannot manage billing	Default role for most paid seats.
Developer	Their own Evidence Ledger and standup drafts; team-level dashboard in read-only view	Two-way transparency — sees the same data judged about them.
Client (Portal-only)	Read-only access to a single white-labeled Client Proof-of-Work Portal; no access to raw code, dashboards, or other workspaces	Issued via scoped, revocable portal links — not a full account by default.

Implementation direction

- **Sign-in:** GitHub OAuth as the primary login (matches the product's identity), email/password as fallback.
- **Session & tokens:** short-lived JWT access tokens + refresh tokens; workspace ID embedded in every token to enforce tenant isolation at the API layer, not just the UI.
- **Authorization:** role-based access control (RBAC) enforced server-side on every API route — never trust client-side role checks alone.
- **Enterprise tier:** SSO/SAML (Okta, Azure AD) and SCIM provisioning, gated to the Enterprise plan.
- **Client portal links:** signed, expiring, revocable URLs scoped to one report or one portal — no shared credentials with clients.
- **Audit logging:** every privileged action (role change, billing change, integration connect/disconnect, data export) written to an immutable AuditLog table.

7. Super-Admin Dashboard (Platform Operator View)

This is your own internal cockpit — not visible to any customer — for monitoring the health of VeriSprint itself across every workspace.

Core panels

Panel	What it shows
System health	API uptime/latency, queue depth and processing lag, LLM call success/failure rate, webhook delivery status per workspace.
Error monitoring	Live error feed (ingestion failures, LLM timeouts, failed webhook deliveries, auth errors) with severity, affected workspace, and stack trace/log link.
Usage analytics	Active workspaces, active users, commits/PRs analyzed per day, feature adoption (which Phase 2/3 features are actually used), Repo Chat query volume.
Billing & revenue overview	MRR, plan distribution, upgrades/downgrades/churn, failed payments, trial-to-paid conversion.
Workspace inspector	Drill into any single customer workspace (with clear audit logging of the access) to debug a specific customer issue — connected repos, recent jobs, recent errors.
LLM cost & usage	Token spend per workspace, cost per analyzed commit, anomaly flags for runaway usage — critical given the cost-control notes in Section 10.
Feature flags & rollout control	Turn Phase 2/3 features on per workspace or per plan tier without a full deploy; kill-switch for any feature misbehaving in production.
Support tools	Impersonate-with-audit-log for support debugging, resend failed webhooks, manually retrigger a stuck analysis job.
Alerting	Pages you (Slack/email/SMS) on error-rate spikes, queue backlog, LLM provider outages, or payment-processor failures.

Why this matters now, specifically

Once real customers are connecting real repos, failures happen silently unless something is watching: a stuck webhook means a customer's dashboard just goes stale with no visible error on their end. The Super-Admin Dashboard is what lets you catch that before a customer does — which matters directly for the trust-based adoption this product depends on.

8. Public Pages & Marketing Site

Direct answer: yes, public pages are still necessary — more so now that this is production, not less.

An MVP demoed live in a meeting can survive without a public site because you're standing in the room explaining it. A production product cannot: every self-serve signup, every GitHub App install, every "someone forwarded this link" moment depends on a public page existing to land on. This is also directly tied to the go-to-market motion already committed to in Section 2 ("narrow, fast, self-serve, no sales call") — that promise is unkeepable without a public site.

Minimum required public pages

Page	Purpose
Landing page	Explains the one-line pitch, shows the Evidence Ledger visually, primary signup CTA.
Pricing page	Transparent, self-serve tiers (matches the Swarmia-style transparent pricing decision in Section 3).
Sign up / Log in	GitHub OAuth entry point — the actual funnel start.
Product/feature pages	Short pages per major feature (Evidence Ledger, Repo Chat, Client Portal) for SEO and sales-enablement links.
Docs / Help Center	Setup guides, FAQ, integration instructions — reduces support load directly.
Changelog / Blog	Builds trust and SEO; also where the free standup-summary tool (top-of-funnel wedge) gets distributed.
Status page	Public system-status page (can be fed by the same health data as the Super-Admin Dashboard) — standard trust signal for any dev tool.
Legal	Privacy Policy, Terms of Service, Data Processing Agreement — required before any real customer connects a real GitHub repo.
Security page	Explains read-only scopes, no-training-on-customer-code policy — directly answers the objection enterprise buyers raise first.

Practical note

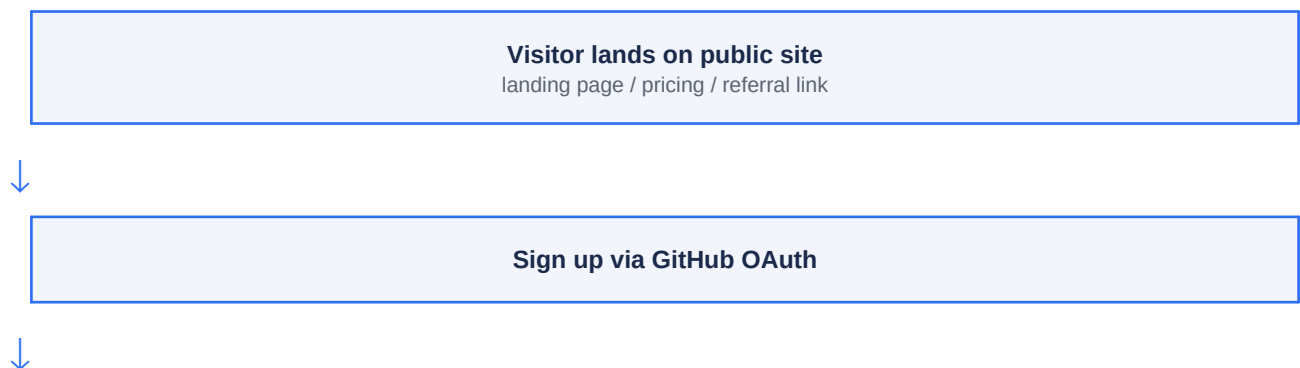
A GitHub App listing in the GitHub Marketplace also requires a public homepage and privacy policy URL as part of GitHub's own submission requirements — so the public site isn't just a marketing nice-to-have here, it's a functional dependency for the GitHub integration itself going fully public.

9. Workflows & System Flows

Flow A — Core Verification Loop



Flow B — New Customer Onboarding (Public Site → Active Workspace)



Install GitHub App, select repos
read-only scopes



Backfill + first analysis run



Workspace active
Super-Admin Dashboard logs the new workspace and its first health signals

Flow C — Error Path (Super-Admin visibility)

Webhook or LLM call fails



Error logged with workspace ID + severity



Super-Admin Dashboard error feed + alert

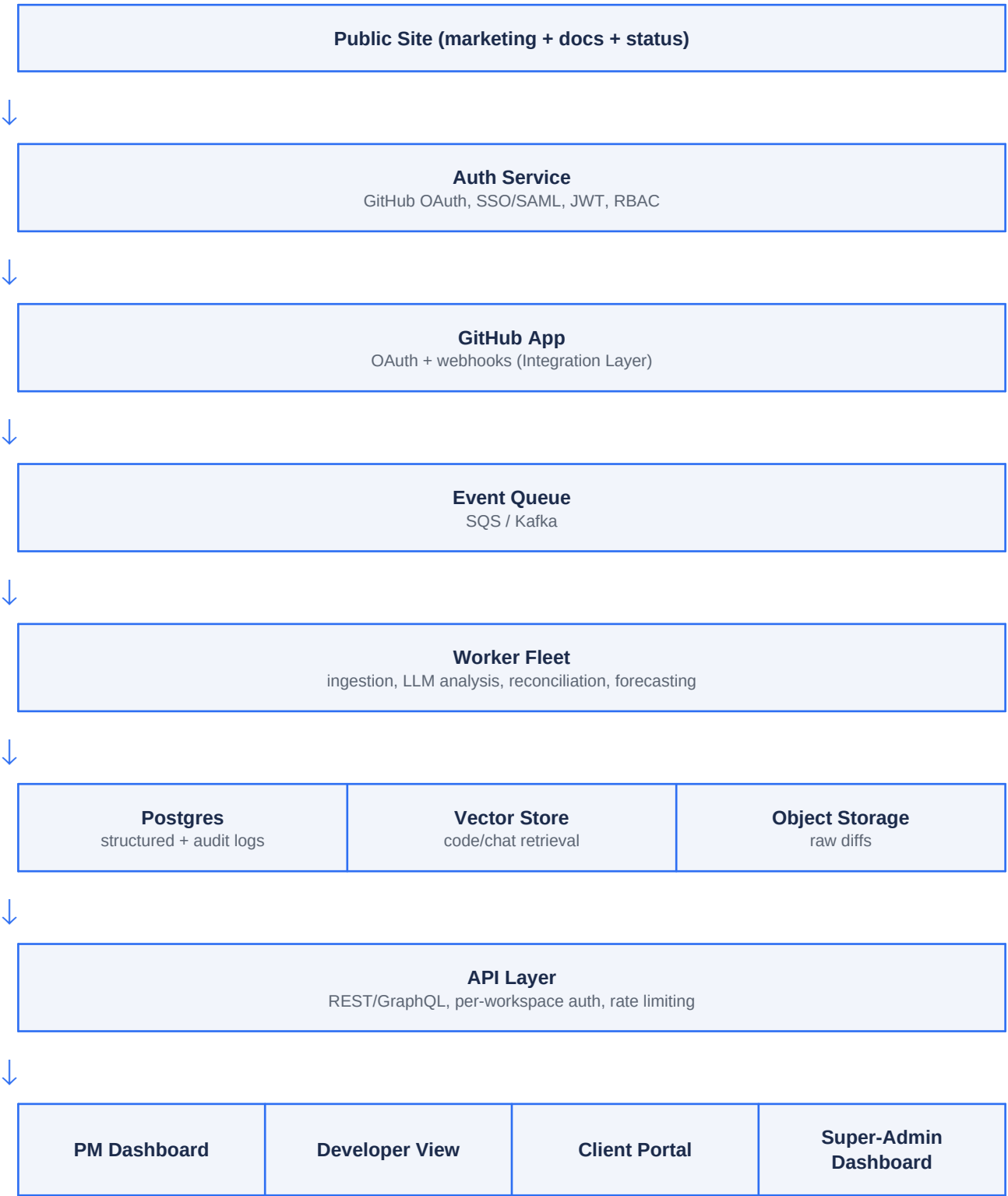


Retry / manual retrigger from Workspace Inspector



Customer dashboard recovers
ideally before the customer notices

10. Production Architecture



Component detail (additions from v2)

Layer	Technology direction	Purpose
Public site	Next.js static/SSR site, separate deploy from the app	Marketing, docs, pricing, signup entry, status page; required for GitHub App listing.
Auth service	GitHub OAuth + SSO/SAML (enterprise), JWT + refresh tokens, RBAC middleware	Tenant isolation and role enforcement on every API call.
Monitoring/observability	Structured logging + error tracking (e.g. Sentry-style) + metrics pipeline feeding the Super-Admin Dashboard	Powers Section 7 in real time.
Billing	Stripe (or equivalent) subscription billing, webhook-synced to Workspace plan state	Drives plan gating for Phase 2/3 features and the Super-Admin billing panel.
Forecasting service	Separate ML service consuming historical Evidence Ledger data per workspace	Powers Delivery Forecast and Risk Radar (Allstacks parity).
Feature flags	LaunchDarkly-style flag service or a lightweight in-house table	Per-workspace/per-plan feature gating, controlled from the Super-Admin Dashboard.

11. Core Data Model

Additions to the v2 data model to support auth, super-admin operations, and public-site conversion tracking.

Entity	Key fields	Notes
User	id, email, github_id, role, workspace_id, created_at, last_login_at	One record per human; role enforced server-side.
Workspace	id, name, plan_tier, github_installation_id, status, mrr	Tenant boundary; unchanged from v2, extended with billing state.
AuditLog	id, actor_user_id, action, target_type, target_id, workspace_id, timestamp	Immutable record of every privileged action — required for Super-Admin trust and compliance mode.
ErrorEvent	id, workspace_id (nullable), source, severity, message, stack_ref, resolved_at	Feeds the Super-Admin error monitoring panel.
SystemMetric	id, metric_name, value, workspace_id (nullable), recorded_at	Time-series for uptime, latency, queue depth, LLM cost.
Subscription	id, workspace_id, plan_tier, status, mrr, renewed_at, payment_provider_ref	Billing state; drives feature gating and the Super-Admin revenue panel.
FeatureFlag	id, key, enabled_globally, enabled_workspace_ids[]	Per-workspace/per-plan rollout control.
ClientPortalLink	id, workspace_id, report_id, token, expires_at, revoked	Scoped, revocable client portal access — not a full account.
EvidenceItem / ConfidenceScore / ReconciliationFlag / StandupEntry / ChatQuery	(unchanged from v2)	Core verification primitives, see Master Spec v2 for full field list.

12. Pricing & Packaging

Tier	Price direction	Includes
Free	\$0	Auto-drafted standup generator for individual developers.
Team	\$12–20 / dev / month	Evidence Ledger, reconciliation, mismatch alerts, DORA panel, Slack digest.
Growth	\$25–35 / dev / month	AI Repo Chat, sprint rollups, Risk Radar, AI Contribution Tracker, Investment Allocation.
Agency / Client-Facing	Custom add-on	White-labeled Client Proof-of-Work Portal, investor update generator.
Enterprise	Custom	SSO/SAML, on-prem LLM, compliance/audit mode, cost capitalization, multi-repo intelligence, dedicated support.

13. Build Brief for Engineering

Updated build order, incorporating auth, super-admin, and public site as first-class production requirements.

Stack additions since v2

- **Auth:** NextAuth.js or Clerk/Auth0 for GitHub OAuth + SSO, backed by the RBAC middleware described in Section 6.
- **Billing:** Stripe, webhook-synced to the Subscription table.
- **Observability:** Sentry (or equivalent) for error tracking, a metrics pipeline (e.g. Prometheus/Grafana or a hosted equivalent) feeding SystemMetric.
- **Public site:** separate Next.js deploy from the authenticated app, sharing the design system.

Production build order

- **Step 1:** Auth service + RBAC across all five roles; workspace tenant isolation enforced server-side.
- **Step 2:** AuditLog wired into every privileged action from day one, not retrofitted later.
- **Step 3:** ErrorEvent + SystemMetric pipeline, minimal Super-Admin Dashboard (health + error feed first, billing/usage panels next).
- **Step 4:** Billing integration (Stripe) + Subscription-driven feature gating via FeatureFlag.
- **Step 5:** Public site: landing, pricing, signup entry, docs, legal, status page — required before any public GitHub App listing.
- **Step 6:** Layer in Phase 2 competitor-parity features (DORA panel, Blocker Nudge Bot, AI Contribution Tracker, Investment Allocation) behind feature flags.
- **Step 7:** Client Portal with scoped ClientPortalLink tokens; Phase 3 features behind Enterprise-gated flags.

Definition of done for production launch

- A new user can sign up on the public site, connect GitHub, and reach first insight without any manual intervention.
- Super-Admin Dashboard shows live health and error data for at least one real workspace.
- A simulated failure (kill a worker mid-job) is visible in the Super-Admin error feed within seconds and does not silently corrupt customer data.
- Billing upgrade/downgrade correctly gates Phase 2 features in real time.
- Public site legal pages (Privacy, Terms, Security) are live before any external customer connects a real repository.

14. Meeting Talking Points

- **Open with what's unchanged:** the Evidence Ledger and Claimed-vs-Shipped verification loop are still the core identity — everything added this round extends outward from that core.
- **Show the parity matrix (Section 3) directly:** name Jellyfish, LinearB, GitClear, Swarmia, and Allstacks, and state plainly that every headline feature across all five now has a home on the roadmap.
- **Walk through the Super-Admin Dashboard as operational maturity:** this signals the product is being run like a real company, not just a demo — important for anyone evaluating this as an investment or early customer.
- **Address the public-site question head-on if asked:** production without a public site can't fulfill the self-serve, no-sales-call promise that is core to the go-to-market strategy, and it's a hard requirement for the GitHub App to go publicly listed.
- **Close with the phased roadmap:** MVP is live, Phase 2 brings the bulk of competitor parity, Phase 3 brings enterprise-grade depth — a credible, sequenced path rather than an all-at-once promise.

End of document.